

Chapter 10

網頁排版佈局 - CSS

FLEXBOX & GRID

HORAZON

互動媒體設計 (一學期)

為什麼需要排版系統？

在很多年前，要讓兩個東西「並排」，我們得用 `<table>` 或是 `float` 這些很髒的手段。現在，CSS 提供了兩套強大的排版系統，讓你可以**隨心所欲**地控制與排列網頁元素：

1. **Flexbox**：處理單行或單列的排列 (例如：導覽列)。
2. **Grid**：處理整個頁面的網格佈局 (例如：照片牆、儀表板)。

PART 1: FLEXBOX

Flexbox 的核心觀念只有兩個角色：

- Flex Container：爸爸。
- Flex Item：兒子。

只要在爸爸身上設定 `display: flex`，兒子們就會乖乖聽話。

```
.container {  
  display: flex; /* 兒子們會變成橫排 */  
}
```

1. 主軸對齊

決定兒子們在**橫向**怎麼排。

屬性：`justify-content`

- `flex-start`：靠左 (預設)。
- `flex-end`：靠右。
- `center`：置中。
- `space-between`：左右推到底，中間平均分配 (導覽列最常用！)。
- `space-around`：每個兒子周圍都有平均的留白。

2. 交錯軸對齊

決定兒子們在**縱向**怎麼排。

屬性：`align-items`

- `stretch`：拉長填滿 (預設)。
- `flex-start`：靠上。
- `flex-end`：靠下。
- `center`：垂直置中 (最常用！)。

聖杯佈局 (完全置中)：

```
.box { display: flex; justify-content: center; align-items: center; }
```

3. 排列方向與換行

- **flex-direction** :
 - **row** : 橫排 (預設, 左->右)。
 - **column** : 直排 (上->下, 手機版常用)。
 - **row-reverse** : 反向橫排 (右->左)。
- **flex-wrap** :
 - **nowrap** : 死都不換行 (預設, 空間不夠會擠壓兒子)。
 - **wrap** : 空間不夠就自動換行。

FLEX 項目屬性

這些是寫在兒子身上的。

- `flex-grow` :
 - `flex-grow: 1;` -> 有剩餘空間就放大填滿。
 - 如果所有兒子都設 1，大家均分空間。
- `flex-shrink` :
 - `flex-shrink: 0;` -> 空間不夠也不准縮小 (保持原大小)。
- `order` :
 - 不用改 HTML 順序，直接用 CSS 改變排列順序。
 - `order: 1` 會排在 `order: 0` 後面。

實戰練習：導覽列

我們用 Flexbox 做一個經典的導覽列：

```
nav {  
  display: flex;  
  justify-content: space-between; /* Logo左，選單右 */  
  align-items: center;           /* 垂直置中 */  
  padding: 20px;  
  background: #333;  
}  
.menu {  
  display: flex;  
  gap: 20px; /* 選單項目之間的距離 */  
}
```

PART 2: CSS GRID

如果 Flexbox 是一條線，那 Grid 就是一張棋盤。

它是最強大的排版系統，可以輕鬆畫出兩欄、三欄、甚至複雜的報紙版面。

```
.container {  
  display: grid;  
  /* 定義三欄：200px, 自動, 100px */  
  grid-template-columns: 200px auto 100px;  
}
```

GRID 的單位：FR

fr 是 Grid 專用的單位，代表「剩餘空間的比例」。

```
.container {  
  display: grid;  
  /* 切成三等份 (1:1:1) */  
  grid-template-columns: 1fr 1fr 1fr;  
  /* 第一欄佔 1 份，第二欄佔 2 份，第三欄佔 1 份 */  
  grid-template-columns: 1fr 2fr 1fr;  
  
  gap: 20px; /* 格子之間的溝槽 */  
}
```

這種寫法比用 % 計算寬度快多了！

GRID 區域

這招超像在畫 ASCII Art，非常直觀！

```
.container {  
  display: grid;  
  grid-template-areas:  
    "header header header"  
    "sidebar main main"  
    "footer footer footer";  
}  
.header { grid-area: header; }  
.sidebar { grid-area: sidebar; }  
.main { grid-area: main; }  
.footer { grid-area: footer; }
```

RWD 響應式排版

配合 Media Query，我們可以輕鬆改變佈局。

電腦版 (3欄)：

```
.cards {  
  display: grid;  
  grid-template-columns: 1fr 1fr 1fr;  
}
```

手機版 (1欄)：

```
@media (max-width: 768px) {  
  .cards {  
    grid-template-columns: 1fr; /* 變成直的一排 */  
  }  
}
```

FLEXBOX VS GRID：怎麼選？

特性	Flexbox	Grid
維度	一維 (線)	二維 (面)
對齊基礎	內容優先	版面優先
適合場景	導覽列、按鈕組、卡片內的排版	整個網頁的架構、照片牆、複雜儀表板
學習曲線	簡單	稍微複雜

實作練習：HOLY GRAIL LAYOUT

請做出經典的網站架構：

1. **Header**：置頂，橫向填滿。
2. **Main Content**：中間三欄式 (左側邊欄、主內容、右側邊欄)。
3. **Footer**：置底，橫向填滿。

要求：

- 使用 Grid 劃分大區域。
- Header 裡面的 Logo 和選單使用 Flexbox 排列。
- 在手機版時，側邊欄要變到主內容下方 (變成單欄)。

覺得很難記？玩遊戲學吧！

1. **Flexbox Froggy** (青蛙過街)

- 用 css code 幫青蛙跳到荷葉上。

2. **Grid Garden** (種蘿蔔)

- 用 grid code 幫蘿蔔澆水。

這兩個遊戲玩通關，你的排版功力就超過 80% 的工程師了。

下週預告

網頁漂亮是漂亮，但它是「死」的。
我們要賦予它「邏輯」與「互動」。

程式邏輯入門 - JavaScript 基礎

- 變數 (`var` , `let` , `const`)
- 判斷式 (`if...else`)
- 迴圈 (`for`)
- 函式 (`function`)

準備好你的邏輯腦，下週見！

